Trip Planner: End-to-End Build Guide

A step-by-step plan for the Flam Frontend Internship Assignment. Target: a clean, solid core in about 8 hours, with failure handling that stands out.

1. What you are optimizing for

Area	Weight	What wins it here
React and frontend architecture	25%	Small components, a reducer for itinerary state, custom hooks, no prop-drilling mess
AI integration and data handling	25%	Backend proxy (key never in the browser), strict JSON schema, validation layer
Handling bad AI output	20%	Malformed JSON, wrong shape, empty, slow, failed, stale responses, all handled
UI/UX and product sense	15%	Expand, remove, reorder, undo, live budget, mobile-friendly
Communication and understanding	15%	Honest README, small commits, and you can explain every line in the interview

Rule of thumb from the brief: **a clean, solid core beats a pile of half-working features.** Do not exceed 8 hours of work; if you run out, stop and write what you would do next.

2. Scope

Must have

- Free-form trip input with example prompt chips
- Backend proxy calling a real LLM, returning JSON
- Zod validation and normalization before rendering
- Day tabs, expandable stop cards, remove stop (with undo), reorder stops (drag and drop plus up/down buttons)
- Live budget total, packing checklist
- Loading, error (with retry), and empty states; stale-response protection
- Mobile-friendly layout

- README with setup, AI-usage note, limitations, time spent

Stretch, in this order

1. Refinement loop ("make day 2 cheaper" edits the current plan)
2. Save and reload via localStorage
3. Dark mode and light animation
4. Different block types (stop, checklist, budget chart)

Skip streaming unless everything else is done.

3. Architecture

```
Browser (React)  --POST /api/plan-->  Express proxy  --->  LLM API
   |                                      |  holds API key, timeout, JSON
extraction
   |  parsePlan() with Zod: validate, salvage, add ids
   v
 useGenerate (request lifecycle)  ->  useItinerary (reducer + undo)  ->
components
```

Repo layout:

```
trip-planner/
  server/index.js        # proxy, timeout, mock modes
  server/prompt.js       # system prompt
  src/schema.js          # Zod + parsePlan
  src/hooks/useGenerate.js
  src/hooks/useItinerary.js
  src/itineraryReducer.js
  src/components/        # TripForm, ItineraryView, DayColumn, StopCard,
                         # BudgetSummary, PackingList, ErrorState,
EmptyState, Skeleton
  src/App.jsx  src/main.jsx  src/styles.css
  .env.example  README.md
```

4. Step 0: Setup (30 min)

```
npm create vite@latest trip-planner -- --template react
cd trip-planner
npm i zod express dotenv @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
npm i -D concurrently vitest
```

`package.json` scripts (the brief says `npm install && npm start` must work):

```
"scripts": {
  "start": "concurrently -k \"npm:server\" \"npm:client\"",
  "server": "node server/index.js",
  "client": "vite",
  "build": "vite build",
  "test": "vitest run"
}
```

`vite.config.js`: proxy API calls to the server.

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: { proxy: { '/api': 'http://localhost:8787' } },
});
```

`.env.example` (commit this, never `.env`; add `.env` to `.gitignore`):

```
GEMINI_API_KEY=
GEMINI_MODEL=gemini-2.0-flash
MOCK=
```

Get a free key from Google AI Studio (or use Groq/OpenRouter). Check the provider's current model names; keep the model in an env var so you can swap it without code changes. Commit message: `chore: scaffold vite app and express proxy`.

5. Step 1: Prompt and server proxy (1 hour)

`server/prompt.js`:

```
export const SYSTEM_PROMPT = `You are a travel planner API. Reply with ONE
JSON object and nothing else.
Shape: {"title": string, "currency": string, "days": [{"label": string,
"stops": [{"name": string, "time": string, "category":
"food|sight|activity|transport|stay|other", "description": string, "estCost":
number}]}], "packingList": [string]}
Rules:
- 1 to 7 days, 3 to 6 stops per day, ordered by time of day.
- Use real, well-known places only. Never include URLs or image links.
- time looks like "9:00 AM". estCost is a number in the traveler's currency
(default INR).
- Respect the user's budget, pace and interests.
- If the message is not a trip request, return {"title": "", "currency":
"INR", "days": [], "packingList": []}.`;
```

`server/index.js`:

```
import 'dotenv/config';
import express from 'express';
import { SYSTEM_PROMPT } from './prompt.js';

const app = express();
app.use(express.json({ limit: '10kb' }));

const MODEL = process.env.GEMINI_MODEL || 'gemini-2.0-flash';
const TIMEOUT_MS = 25000;
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

// Models sometimes wrap JSON in prose or code fences: take first { to last
}.
function extractJson(text) {
  const start = text.indexOf('{');
  const end = text.lastIndexOf('}');
  if (start === -1 || end <= start) throw new Error('no json');
  return JSON.parse(text.slice(start, end + 1));
}

app.post('/api/plan', async (req, res) => {
  const prompt = typeof req.body?.prompt === 'string' ?
req.body.prompt.trim() : '';
  if (prompt.length < 5 || prompt.length > 1000) {
    return res.status(400).json({ error: 'Describe your trip in 5 to 1000
```

```
characters.' });
  }

  // Mock modes: demo every failure path without waiting for the model to
misbehave.
  const mock = process.env.MOCK;
  if (mock === 'jitter') await sleep(Math.random() * 6000); // out-of-order
responses
  if (mock === 'slow') await sleep(40000); // trips the client timeout
  if (mock === 'bad-json') return res.status(502).json({ error: 'The AI
returned malformed JSON.' });
  if (mock === 'bad-shape') return res.json({ plan: { title: 'Oops', days:
'nope' } });
  if (mock === 'empty') return res.json({ plan: { title: 'Nothing', days: []
} });
  if (mock === 'fail') return res.status(500).json({ error: 'Server error.'
});

  const ctrl = new AbortController();
  const timer = setTimeout(() => ctrl.abort(), TIMEOUT_MS);
  try {
    const r = await fetch(

`https://generativelanguage.googleapis.com/v1beta/models/${MODEL}:generateCon
tent`,
      {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', 'x-goog-api-key':
process.env.GEMINI_API_KEY },
        body: JSON.stringify({
          systemInstruction: { parts: [{ text: SYSTEM_PROMPT }] },
          contents: [{ role: 'user', parts: [{ text: prompt }] }],
          generationConfig: { responseMimeType: 'application/json',
temperature: 0.7 },
        }),
        signal: ctrl.signal,
      }
    );
    if (!r.ok) return res.status(502).json({ error: 'The AI provider returned
an error.' });
    const data = await r.json();
    const text = data?.candidates?.[0]?.content?.parts?.[0]?.text ?? '';
    if (!text) return res.status(502).json({ error: 'The AI returned an empty
```

```
response.' });
    try {
      return res.json({ plan: extractJson(text) });
    } catch {
      return res.status(502).json({ error: 'The AI returned malformed JSON.'
});
    }
  } catch (e) {
    if (e.name === 'AbortError') return res.status(504).json({ error: 'The AI
took too long.' });
    return res.status(500).json({ error: 'Server error.' });
  } finally {
    clearTimeout(timer);
  }
});

app.listen(process.env.PORT || 8787, () => console.log('API on :8787'));
```

Why this design: the key stays server-side; input is length-checked; the timeout protects you from hanging requests; JSON mode (`responseMimeType`) reduces malformed output but is **not** trusted, so the client still validates. Know what each line does; they will ask.

Commit: `feat(server): proxy with timeout, json extraction, mock modes`.

6. Step 2: Schema and validation (1 hour)

The model is unpredictable, so the UI must only ever see clean data. `src/schema.js` :

```
import { z } from 'zod';

export const CATEGORIES = ['food', 'sight', 'activity', 'transport', 'stay',
'other'];

const Stop = z.object({
  name: z.string().min(1),
  time: z.string().catch(''),
  category: z.string().catch('other').transform((c) =>
(CATEGORIES.includes(c.toLowerCase()) ? c.toLowerCase() : 'other')),
  description: z.string().catch(''),
  estCost: z.coerce.number().min(0).catch(0),
});
```

```
const Top = z.object({
  title: z.string().catch('Your trip'),
  currency: z.string().catch('INR'),
  days: z.array(z.any()),
  packingList: z.array(z.string()).catch([]),
});

// Returns { ok: true, plan, dropped } or { ok: false, error: 'shape' |
'empty' }
export function parsePlan(raw) {
  const top = Top.safeParse(raw);
  if (!top.success) return { ok: false, error: 'shape' };

  let dropped = 0;
  const days = top.data.days
    .map((d, i) => {
      const rawStops = Array.isArray(d?.stops) ? d.stops : [];
      const stops = [];
      for (const s of rawStops) {
        const r = Stop.safeParse(s);
        if (r.success) stops.push({ ...r.data, id: crypto.randomUUID() });
        else dropped++;
      }
      const label = typeof d?.label === 'string' && d.label ? d.label : `Day
${i + 1}`;
      return { id: crypto.randomUUID(), label, stops };
    })
    .filter((d) => d.stops.length > 0);

  if (days.length === 0) return { ok: false, error: 'empty' };
  const { title, currency, packingList } = top.data;
  return { ok: true, plan: { title, currency, days, packingList }, dropped };
}
```

Key ideas to explain: **validate, then salvage** (one broken stop should not kill the whole plan), **generate IDs on the client** (never trust model IDs for React keys or drag-and-drop), **normalize** (unknown category becomes `other`, bad cost becomes 0).

Commit: `feat: zod schema with salvage and normalization`.

7. Step 3: Request lifecycle hook (1 hour)

This is where most of the "bad output" and "stale response" score comes from. `src/hooks/useGenerate.js`:

```
import { useCallback, useRef, useState } from 'react';
import { parsePlan } from '../schema';

const CLIENT_TIMEOUT_MS = 30000;

export function useGenerate() {
  const [state, setState] = useState({ status: 'idle', plan: null, error:
null, dropped: 0 });
  const ctrlRef = useRef(null);
  const latestId = useRef(0);

  const generate = useCallback(async (prompt) => {
    ctrlRef.current?.abort(); // cancel the previous request
    const ctrl = new AbortController();
    ctrlRef.current = ctrl;
    const id = ++latestId.current; // only the newest id may write state
    const timer = setTimeout(() => ctrl.abort(), CLIENT_TIMEOUT_MS);
    setState((s) => ({ ...s, status: 'loading', error: null }));

    const run = async (attempt) => {
      const res = await fetch('/api/plan', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ prompt }),
        signal: ctrl.signal,
      });
      const body = await res.json().catch(() => null);
      if (!res.ok) throw new Error(body?.error || 'Something went wrong.
Please try again.');
      const parsed = parsePlan(body?.plan);
      if (!parsed.ok && parsed.error === 'shape' && attempt < 1) return
run(attempt + 1); // one auto-retry
      return parsed;
    };

    try {
      const parsed = await run(0);
      if (id !== latestId.current) return; // stale: ignore
```

```
      if (parsed.ok) setState({ status: 'success', plan: parsed.plan, error:
null, dropped: parsed.dropped });
      else if (parsed.error === 'empty') setState({ status: 'empty', plan:
null, error: null, dropped: 0 });
      else setState({ status: 'error', plan: null, error: 'The AI sent a plan
we could not read.', dropped: 0 });
    } catch (e) {
      if (id !== latestId.current) return; // aborted by a newer request:
ignore
      const msg = e.name === 'AbortError' ? 'This is taking too long.' :
e.message;
      setState({ status: 'error', plan: null, error: msg, dropped: 0 });
    } finally {
      clearTimeout(timer);
    }
  }, []);

  const cancel = useCallback(() => {
    latestId.current++;
    ctrlRef.current?.abort();
    setState((s) => ({ ...s, status: s.plan ? 'success' : 'idle' }));
  }, []);

  return { state, generate, cancel };
}
```

Two layers protect against stale responses: `AbortController` cancels the old network request, and the `latestId` check guarantees an old response that still arrives cannot overwrite newer state. Be ready to explain why you need both.

Commit: `feat: useGenerate with abort, timeout, retry, stale guard`.

8. Step 4: Itinerary state (45 min)

Pure functions plus a reducer make this easy to test and easy to explain. `src/itineraryReducer.js`:

```
export const initial = { present: null, past: [] };

const commit = (s, present) => ({ present, past: [...s.past,
s.present].slice(-20) });
```

```
export function removeStop(plan, stopId) {
  return { ...plan, days: plan.days.map((d) => ({ ...d, stops:
d.stops.filter((s) => s.id !== stopId) })) };
}

export function moveStop(plan, stopId, toDayId, toIndex) {
  let moved = null;
  const without = plan.days.map((d) => ({
    ...d,
    stops: d.stops.filter((s) => {
      if (s.id === stopId) { moved = s; return false; }
      return true;
    }),
  }));
  if (!moved) return plan;
  return {
    ...plan,
    days: without.map((d) =>
      d.id === toDayId ? { ...d, stops: [...d.stops.slice(0, toIndex), moved,
...d.stops.slice(toIndex)] } : d
    ),
  };
}

export function reducer(state, action) {
  switch (action.type) {
    case 'LOAD':
      return { present: action.plan, past: [] };
    case 'REMOVE_STOP':
      return commit(state, removeStop(state.present, action.stopId));
    case 'MOVE_STOP':
      return commit(state, moveStop(state.present, action.stopId,
action.toDayId, action.toIndex));
    case 'UNDO': {
      if (state.past.length === 0) return state;
      return { present: state.past[state.past.length - 1], past:
state.past.slice(0, -1) };
    }
    default:
      return state;
  }
}
```

`src/hooks/useItinerary.js`:

```
import { useEffect, useReducer } from 'react';
import { reducer, initial } from '../itineraryReducer';

export function useItinerary(generatedPlan) {
  const [state, dispatch] = useReducer(reducer, initial);
  useEffect(() => {
    if (generatedPlan) dispatch({ type: 'LOAD', plan: generatedPlan });
  }, [generatedPlan]);
  return { plan: state.present, dispatch, canUndo: state.past.length > 0 };
}
```

Never mutate state; every action returns new objects. Remember that removing the last stop of a day leaves an empty day: decide whether to show "No stops yet, drag one here" (recommended) and handle it.

Budget total is derived, not stored: `plan.days.flatMap((d) => d.stops).reduce((sum, s) => sum + s.estCost, 0)`.

Commit: `feat: itinerary reducer with undo`.

9. Step 5: UI components (1.5 hours)

App states

```
export default function App() {
  const [prompt, setPrompt] = useState('');
  const { state, generate, cancel } = useGenerate();
  const { plan, dispatch, canUndo } = useItinerary(state.plan);

  return (
    <main className='app'>
      <TripForm value={prompt} onChange={setPrompt} onSubmit={() =>
generate(prompt)} loading={state.status === 'loading'} />
      {state.status === 'idle' && <EmptyHint onPick={setPrompt} />}
      {state.status === 'loading' && <Skeleton onCancel={cancel} />}
      {state.status === 'error' && <ErrorState message={state.error} onRetry=
{() => generate(prompt)} />}
      {state.status === 'empty' && <EmptyState />}
      {state.status === 'success' && plan && (
        <ItineraryView plan={plan} dispatch={dispatch} canUndo={canUndo}
dropped={state.dropped} />
      )}
    </main>
  );
}
```

StopCard (expand, drag handle, remove, accessible buttons):

```
import { useState } from 'react';
import { useSortable } from '@dnd-kit/sortable';
import { CSS } from '@dnd-kit/utilities';

export default function StopCard({ stop, currency, onRemove, onUp, onDown })
{
  const [open, setOpen] = useState(false);
  const { attributes, listeners, setNodeRef, transform, transition,
isDragging } = useSortable({ id: stop.id });
  const style = { transform: CSS.Transform.toString(transform), transition,
opacity: isDragging ? 0.6 : 1 };

  return (
    <li ref={setNodeRef} style={style} className='stop'>
      <button className='handle' aria-label={`Drag ${stop.name}`}
{...attributes} {...listeners}>⋮⋮</button>
      <button className='stop-main' aria-expanded={open} onClick={() =>
setOpen((o) => !o)}>
        <span className={`tag tag-${stop.category}`}>{stop.category}</span>
```

```
        <strong>{stop.name}</strong>
        <span className='time'>{stop.time}</span>
      </button>
      <button aria-label={`Move ${stop.name} up`} onClick={onUp}>↑</button>
      <button aria-label={`Move ${stop.name} down`} onClick=
{onDown}>↓</button>
      <button aria-label={`Remove ${stop.name}`} onClick={() =>
onRemove(stop.id)}>×</button>
      {open && <p className='details'>{stop.description} · {currency}
{stop.estCost}</p>}
    </li>
  );
}
```

Drag and drop across days (one `DndContext`, one `SortableContext` per day, and each `DayColumn` also uses `useDroppable({ id: day.id })` so empty days accept drops):

```
const sensors = useSensors(
  useSensor(PointerSensor, { activationConstraint: { distance: 8 } }),
  useSensor(TouchSensor, { activationConstraint: { delay: 200, tolerance: 6 }
}),
  useSensor(KeyboardSensor, { coordinateGetter: sortableKeyboardCoordinates
})
);

function onDragEnd({ active, over }) {
  if (!over || active.id === over.id) return;
  const overDay = plan.days.find((d) => d.id === over.id);
  const targetDay = overDay ?? plan.days.find((d) => d.stops.some((s) => s.id
=== over.id));
  if (!targetDay) return;
  const toIndex = overDay ? targetDay.stops.length :
targetDay.stops.findIndex((s) => s.id === over.id);
  dispatch({ type: 'MOVE_STOP', stopId: active.id, toDayId: targetDay.id,
toIndex });
}
```

Build order: first plain list with expand and remove, then **same-day** reorder, then cross-day. The up/down buttons are your fallback if drag-and-drop gives trouble on touch, and they are good accessibility anyway.

Also build: `BudgetSummary` (total and per-day), `PackingList` (checkboxes in local state), an Undo button shown when `canUndo`, and a small notice when `dropped > 0` ("Some stops were skipped because the AI sent invalid data").

Commits: `feat: stop cards with expand/remove`, `feat: drag and drop reorder`, `feat: budget and packing list`.

10. Step 6: States, mobile, accessibility (45 min)

- **Loading:** skeleton cards and a Cancel button; disable the submit button while loading.
- **Error:** plain-language message, a Retry button that reuses the same prompt, and the input text preserved.
- **Empty:** dedicated message for "AI returned no plan" plus example prompts.
- **Mobile:** `<meta name='viewport' content='width=device-width, initial-scale=1'>`; single column; horizontally scrollable day tabs; tap targets at least 44px; `touch-action: none` on the drag handle only (so the page still scrolls); test with `vite --host` on your real phone, not only DevTools.
- **Dark mode (cheap):** CSS variables plus `@media (prefers-color-scheme: dark)`.
- **A11y:** real `<button>`s, `aria-expanded`, labels on icon buttons, visible focus outline.

11. Step 7: Tests (30 min)

Test the logic, not the pixels. A few Vitest tests for `parsePlan` and the reducer are enough to show discipline:

```
import { describe, it, expect } from 'vitest';
import { parsePlan } from '../src/schema';

describe('parsePlan', () => {
  it('rejects wrong shape', () => {
    expect(parsePlan({ title: 'x', days: 'nope' })).toEqual({ ok: false,
error: 'shape' });
  });
  it('reports empty days', () => {
    expect(parsePlan({ title: 'x', days: [] }).error).toBe('empty');
  });
  it('drops a bad stop but keeps the rest', () => {
    const r = parsePlan({ title: 'x', days: [{ stops: [{ name: 'A' }, { nope:
1 }] }] });
```

```
    expect(r.ok).toBe(true);
    expect(r.plan.days[0].stops).toHaveLength(1);
    expect(r.dropped).toBe(1);
  });
  it('normalizes unknown category and bad cost', () => {
    const s = parsePlan({ title: 'x', days: [{ stops: [{ name: 'A', category:
'spaceship', estCost: 'abc' }] }] }).plan.days[0].stops[0];
    expect(s.category).toBe('other');
    expect(s.estCost).toBe(0);
  });
});
```

Also test `moveStop` (same day down, same day up, cross-day, to empty day) and `removeStop`. Off-by-one bugs here are exactly what an interviewer might introduce.

Manual failure checklist (run with the `MOCK` env var, then restart the server):

Scenario	How	Expected
Malformed JSON	`MOCK=bad-json`	Friendly error, Retry works
Wrong shape	`MOCK=bad-shape`	Auto-retry once, then error with Retry
Empty plan	`MOCK=empty`	Empty state
Server failure	`MOCK=fail`	Error state
Slow	`MOCK=slow`	Timeout message after 30s, Cancel works
Stale response	`MOCK=jitter`, submit two prompts quickly	Only the latest result ever shows
Offline	Browser DevTools offline	Error state, no crash
Bad key	Wrong `GEMINI_API_KEY`	Error state, key never visible in browser network tab

12. Step 8: Stretch features (only if time remains)

Refinement loop. Add a second input ("make day 2 cheaper"). Send `{ prompt: instruction, currentPlan }` to the server and change the system prompt to: "Here is the current plan as JSON. Apply the user's instruction and return the full updated plan in the same shape." Run the result through the same `parsePlan`. This reuses everything you built, which is a great story for the interview. Keep undo working by dispatching `LOAD` as a new history entry.

Save and reload. Persist `{ prompt, present }` to localStorage in an effect; restore on load and validate with `parsePlan` again (stored data can also be corrupted). Wrap in try/catch.

Block types. Allow `blocks: [{ type: 'stop' | 'checklist' | 'budget' }]` and render with a `switch` on `type`, with an `Unknown block` fallback so a new type never crashes the UI.

13. Step 9: README template

The brief explicitly grades this. Keep it honest and specific.

```
# Trip Planner
Describe a trip in plain words; AI returns structured JSON that becomes an
interactive itinerary (expand, remove, reorder, budget).

## Demo
Link to screen recording and, if deployed, the live URL.

## Setup
1. Node 18+
2. cp .env.example .env and add GEMINI_API_KEY
3. npm install && npm start  (client :5173, API :8787)

## How it works
Diagram or a few lines: React -> Express proxy -> LLM -> Zod validation ->
reducer -> UI.

## Handling bad AI output
List: malformed JSON, wrong shape (salvage + one auto-retry), empty, timeout,
provider error, stale responses (AbortController + request id). Mention the
MOCK modes and how to try them.
```

```
## AI usage note
What I used AI for (for example: boilerplate, CSS, reviewing the reducer) and
what I wrote myself (schema, request lifecycle, reducer). Be honest.

## Known limitations
Examples: no real maps or live prices, costs are AI estimates, the model can
suggest closed or non-existent places, free-tier rate limits, no auth.

## What I would do next
Streaming, refinement loop, saved sessions, e2e tests.

## Time spent
Breakdown that adds up to 8 hours or less.
```

Keep a simple time log as you go so the "time spent" section is truthful.

14. Step 10: Recording, deploy, submit

Screen recording (60 to 90 seconds)

1. Enter a trip prompt, show the skeleton, then the itinerary.
2. Expand a stop, reorder with drag, move across days, remove, then Undo; show the budget changing.
3. Show the phone-sized layout.
4. Show two or three failure modes using `MOCK` (bad-json with Retry, empty, jitter or stale).
5. Briefly show the repo and README.

Deploy (optional but preferred). Add `app.use(express.static('dist'))` to the server, run `npm run build`, and host the single Node service on Render or Railway, with the API key as an environment variable. Expect cold starts on free tiers, which your loading state already handles. Do not commit `.env`.

Submission checklist

- [ ] Public repo, small meaningful commits
- [ ] Fresh clone works with `npm install && npm start`
- [ ] `.env` not committed; no key in the built client (search `dist` for it)
- [ ] README complete, recording linked
- [ ] Under 8 hours, with the time honestly noted

15. Interview preparation

Expect: demo, code walkthrough, review of an AI-generated snippet, fixing a bug they introduce, and adding a small feature.

Questions to be able to answer

- Why a proxy server? (API key safety, validation of input, timeout, rate limiting.)
- Why validate when JSON mode exists? (Models can still return wrong types, missing fields, or empty content; JSON mode guarantees syntax at best.)
- How do you prevent a stale response from overwriting a newer one? (Abort plus request id check.)
- Why `useReducer`? (Several related actions on nested state, pure functions, easy undo and tests.)
- Why generate IDs on the client? (Stable React keys and drag IDs; never trust model output for identity.)
- What happens if one stop is invalid? (Dropped and counted; the rest still renders.)
- What would you change for production? (Rate limiting, caching, streaming, observability, schema versioning.)

Bugs they may introduce: index used as a React key, mutating state in the reducer, off-by-one in `moveStop`, missing `clearTimeout`, removing the `latestId` check, dropping the `catch` fallback. Practice finding each one in your own code.

Small features they may ask for: per-day cost subtotal, filter by category, edit a stop's time, duplicate a stop, "regenerate this day only", copy itinerary as text. Try adding two of these yourself before the interview.

AI-snippet review: look for missing error handling, unvalidated JSON, missing abort or cleanup in effects, and keys or state mistakes.

Rule: **do not ship any line you cannot explain.** If AI wrote a part, read it until you can rewrite it from memory.

16. Schedule

Block	Time	Output
Setup and proxy	1.0h	App runs, model returns JSON

Block	Time	Output
Schema and hook	1.5h	Validated plan, retry, stale guard
Reducer and UI	2.0h	Expand, remove, undo
Drag and drop and budget	1.5h	Reorder across days, totals
States, mobile, dark mode	0.75h	Polished, responsive
Tests	0.5h	Parser and reducer tests
README, recording, submit	0.75h	Done

If you fall behind, cut in this order: dark mode, cross-day drag (keep same-day plus buttons), packing list. Never cut failure handling, README, or the proxy.

17. Resume and application lines

Adapt these to what you actually build (add real numbers only if you measured them):

- Built an AI trip planner (React, Node/Express, Gemini) that converts free-form text into a validated, interactive itinerary with drag-and-drop reordering and undo.
- Designed a resilient AI-output pipeline: server-side proxy with timeouts, Zod schema validation with partial salvage, automatic retry, and request cancellation to prevent stale responses.
- Wrote unit tests for the parser and state reducer; documented AI usage, limitations and failure modes.

In your message to Flam, link the repo, the recording and one line on why you chose the trip planner: it turns generative AI output into tactile, visual interaction, which is what the company builds.